



Quantum Bluff

FINAL PROJECT REPORT

University of Strasbourg

Bachelor of Science in Computer Science (L3)

Supervised project — reference specification: Docs/Cahier_de_charge_6A_PI.pdf

Academic supervisor: *[to be completed by the team]*

Project manager: Omar

Team members: Linda, Massi, Abdoulaye, Soheil, Mohamed, Azra, Yigit

Academic year 2025–2026

Document generated: 14 May 2026

Abstract

Quantum Bluff is a capstone web platform produced for the third-year Bachelor's degree in Computer Science (L3) at the University of Strasbourg. It combines an authoritative Texas Hold'em stack with extended casino-style modules (solo and multiplayer blackjack, slots, European roulette), structured tournaments with ancillary winner markets, social features (friends, invitations, peer loans), and progression systems (XP, daily challenges, leaderboards). This report explains how delivery evolved from an initial V-model framing to an agile-inspired workflow; which engineering tools preserved modularity and testability; and how runtime architecture separates the React client, Express REST endpoints, a Socket.IO gateway, Prisma-backed persistence, and Redis-backed coordination. Security controls (JWT and optional TOTP, segmented rate limits, idempotency on sensitive mutations, anti-cheat hooks) and an automated test harness are summarized alongside appendices that map each major domain to repository artefacts. The product uses **virtual chips** for pedagogical demonstration; it does not implement real-money gambling.

Résumé

Quantum Bluff est une plateforme web temps réel développée dans le cadre du projet de fin d'études de L3 informatique à l'Université de Strasbourg. Elle couvre le poker Texas Hold'em (tables cash, bots, paris latents), des modules type casino (blackjack solo et multijoueur, machine à sous, roulette européenne), des tournois structurés avec marchés annexes sur le vainqueur, des fonctions sociales (amis, invitations, prêts entre joueurs) et de la progression (XP, défis quotidiens, classements). Ce rapport présente l'évolution de la gestion de projet (phase en V initiale puis itérations inspirées du Scrum), les choix technologiques, l'architecture logicielle, la sécurité et les résultats des tests automatisés. Des annexes indexent les chemins du dépôt pour chaque sous-système. Le système repose sur une **monnaie virtuelle** à visée pédagogique et ne constitue pas un service de jeu d'argent réel.

Keywords: multiplayer games, Texas Hold'em, real-time web, WebSocket, TypeScript, React, Node.js, Express, Prisma, PostgreSQL, Redis, Socket.IO, software engineering, security, automated testing.

Mots-clés : jeux multijoueurs, Texas Hold'em, web temps réel, WebSocket, TypeScript, React, Node.js, Express, Prisma, PostgreSQL, Redis, Socket.IO, génie logiciel, sécurité, tests automatisés.

List of abbreviations

API	Application Programming Interface (HTTP/JSON in this project).
CI/CD	Continuous integration and continuous deployment (GitLab CI at repository root).
HUD	Heads-up display: in-table probability and hand-category feedback.
JWT	JSON Web Token used for HTTP and Socket.IO authentication.
ORM	Object-relational mapping (Prisma).
REST	Representational state transfer style for <code>/api/*</code> routes.
RTK	Redux Toolkit and RTK Query for client-side data fetching.
TOTP	Time-based one-time password (optional second factor).
TTL	Time-to-live for cached or in-memory sessions.
UI/UX	User interface and user experience layers in the React SPA.
2FA	Two-factor authentication (TOTP in production paths).

Table of contents

Abstract	i
Résumé (français)	i
List of abbreviations	ii
1 Introduction	1
1.1 Context and Objectives	1
1.2 Scope of this document	1
2 Methodology, traceability, and reading guide	2
2.1 Traceability to the initial specification	2
2.2 Evidence hierarchy	2
2.3 How validation is reported	2
3 Project Evolution	3
3.1 Initial specifications (poker-first)	3
3.2 Scope expansion (casino, tournaments, social, progression)	3
3.3 Why the narrative moved from “catalog” to “appendices”	3
4 Project Management	4
4.1 Initial V-model phase	4
4.2 Transition to an agile-inspired workflow (Jira, Scrum practices)	4
4.3 Planning, coordination, and reporting	4
4.4 Product management mechanics	4
5 Development Tools and Technologies	5
5.1 Frontend and backend	5
5.2 Database and infrastructure	5
5.3 Version control and collaboration	5
5.4 Testing and deployment	5
6 Use of Artificial Intelligence	6
6.1 In-product AI: poker bots	6
6.2 AI-assisted engineering (tooling)	6
6.3 Limits and verification	6
7 Platform Functionalities	7
8 System Architecture	9
8.1 Monorepo layout	9
8.2 Real-time communication	9
8.3 Persistence and integrity	10

8.4	Data model overview (Prisma)	10
8.5	Diagram	11
9	Deployment and operational runbook	12
9.1	Local development stack	12
9.2	Production and desktop delivery	12
10	Security, Testing and Stability	13
10.1	Authentication and anti-cheat	13
10.2	Testing and debugging	14
10.3	Automated test inventory and CI (reference snapshot)	15
10.4	Performance and stability	15
11	Technical Challenges	16
11.1	Implemented solutions	16
12	Ethics, privacy, and academic integrity	19
12.1	Virtual economy and regulatory framing	19
12.2	Personal data	19
12.3	Integrity of authorship and AI assistance	19
13	Individual Contributions	20
13.1	Omar – project management and architecture alignment	20
13.2	Linda – security, real-time features and UI/UX	20
13.3	Other contributors	20
14	Limitations and Future Work	21
14.1	Current limitations	21
14.2	Future improvements	21
15	Conclusion	22
	Acknowledgements	23
A	Repository map and routing	24
A.1	Programmed routing and shell	24
B	Poker engine and cash game	25
B.1	What was programmed	25
B.2	Rules enforced	25
B.3	Further reading	25
C	Bot AI and practice integration	26
C.1	Programmed pipeline	26
C.2	Strength signal and decisions	26

C.3	Further reading	26
D	Probabilities, Quantum HUD, and Monte Carlo helper	27
D.1	Division of responsibilities	27
D.2	Monte Carlo equity estimator	27
D.3	Monte Carlo error budget (HUD analogue to RTP)	27
D.4	Quantum Combi—exact enumeration formula	28
D.5	Quantum Combi in the hand-ranking ladder (server and HUD)	28
E	Hidden betting system	29
E.1	Programmed architecture	29
E.2	Odds and tables (example)	29
E.3	Further reading	29
F	Tournaments and winner side bets	30
F.1	Programmed modules	30
F.2	Further reading	30
G	Blackjack solo and multiplayer	31
G.1	Solo blackjack engine	31
G.2	Further reading	31
H	Slot machine, roulette, and casino wallet	32
H.1	Three-reel slot model	32
H.2	Complete slot RTP (closed form, current constants)	32
H.3	European roulette model	33
H.4	Complete roulette RTP (all proportional inside bets)	33
H.5	Wallet and ledger	33
I	Social graph, invitations, and peer loans	34
I.1	Programmed surfaces	34
I.2	Further reading	34
J	Progression, challenges, and leaderboards	35
J.1	Programmed logic	35
J.2	Further reading	35
K	Authentication, profiles, and i18n	36
K.1	Programmed security and profiles	36
L	Operations, Redis, CI, and packaging	37
L.1	Programmed operations	37
M	External references and compendium	38

List of Figures

- | | | |
|---|--|----|
| 1 | High-level runtime flow (placeholder box; replace with exported diagram if desired). | 11 |
|---|--|----|

List of Tables

1	High-level functional inventory (representative surfaces).	7
2	Automated test inventory (representative snapshot; see <code>Docs/RAPPORT_TESTS.md</code>).	15
3	Individual contributions (template — team must fill before submission).	20

1 Introduction

1.1 Context and Objectives

Quantum Bluff was developed as part of the final third-year Bachelor’s degree project (L3 Computer Science, French academic system) at the University of Strasbourg. The objective was to deliver a **real-time multiplayer gaming platform** with a professional-grade engineering posture: modular architecture, automated testing, observability hooks, security hardening, and collaborative delivery under time constraints.

The **initial product intent** (before scope expansion) was a Texas Hold’em poker platform: practice mode against AI with multiple difficulty levels, multiplayer cash tables (public/private), configurable blinds, minimum buy-in rules, a probability-oriented HUD, and a **hidden side-bet** subsystem. The technical goals included authoritative server-side rules, transactional chip movements, WebSocket-scale event delivery, and operational readiness (health checks, metrics, recovery paths).

Security and reliability were first-class constraints from the outset: JWT authentication, optional TOTP 2FA, anti-cheat and rate-limiting layers, idempotent handling for sensitive mutations, and schema validation on critical API paths.

1.2 Scope of this document

This report summarizes **management evolution, tooling, delivered capabilities, and architecture**. Exhaustive technical narratives (bots, probabilities, tournaments, hidden bets, etc.) are deferred to **Appendices A–M** to avoid repetition. Repository-relative file paths use `\path{...}` notation. The front matter includes an abstract (English and French), a list of abbreviations, the table of contents, and lists of figures and tables.

2 Methodology, traceability, and reading guide

2.1 Traceability to the initial specification

The baseline expectations for the Bachelor project are captured in the repository specification `Docs/Cahier_de_charge_6A_PI.pdf`. The delivered system **extends** that poker-centric charter: all original poker-facing commitments (cash tables, bots, HUD, hidden bets, real-time play) remain grounded in server authority, while additional modules (casino mini-games, tournaments, social lending, progression) were integrated under the same engineering constraints (tests, rate limits, transactional chip movements). Where the PDF and the code diverge, **the implementation and this report defer to the repository as the operational source of truth**, with internal notes under `Docs/detail_logique/` providing rationale.

2.2 Evidence hierarchy

Claims in the body are intentionally **outcome-oriented**. File paths anchor verifiable detail. Long-form auto-generated material (for example `Docs/Rapport_final/RAPPORT_TECHNIQUE_COMPLET.md`) is treated as an *index*: useful for breadth, but not cited as automatically validated against every line of code.

2.3 How validation is reported

Automated suites and manual smoke scenarios are summarized in Section 10 and cross-checked against `Docs/RAPPORT_TESTS.md` and `Docs/rapport-validation.md`. Playwright end-to-end tests live under `client/e2e/`. CI behaviour is defined in `.gitlab-ci.yml`.

3 Project Evolution

3.1 Initial specifications (poker-first)

The early scope targeted: Texas Hold'em cash games; **AI practice** with four difficulty tiers (easy, medium, hard, expert) enforced in `server/src/routes/game.api.routes.ts` and `server/src/routes/bot.routes.ts`; public/private multiplayer rooms with SB/BB configuration and minimum balance rules; **real-time** updates via Socket.IO; a **Quantum HUD** for win-equity and hand-category feedback (`client/src/components/QuantumHUD.tsx`); and **hidden bets** resolved server-side (`server/src/poker/hiddenBets/`).

3.2 Scope expansion (casino, tournaments, social, progression)

The delivered system adds: **Blackjack** solo and **multi-table** lobbies (`client/src/pages/Blackjack.tsx`, `client/src/pages/BlackjackMultiLobby.tsx`, `client/src/pages/BlackjackMultiTable.tsx`); **slot** and **roulette** flows reachable from the mini-games hub (`client/src/pages/MiniGames.tsx`); **tournaments** with waiting room, in-tournament table, results, and **winner prediction side bets** (`client/src/features/tournament/`); **leaderboards**; **XP**, badges, daily challenges, daily login streak; **friends**, invitations, join requests, **peer loans** with interest/repayment hooks; **profile** editing with server-stored avatars; **2FA TOTP**; **feedback** and **player reports**; **wallet ledger**, gift codes, and dev recharge helpers; **i18n** in five locales (`client/src/i18n/locales/`); **tutorial** flow (`client/src/pages/TutorialGame.tsx`, `client/src/features/tutorial/`); **admin web console** and packaged targets (**Electron**, **Capacitor**) per `client/package.json`.

3.3 Why the narrative moved from “catalog” to “appendices”

The codebase grew across **client**, **server**, **database**, and **docs/** technical notes. Listing every subsystem in prose would duplicate `docs/ARCHITECTURE.md` and the structured logic notes under `Docs/detail_logique/`. The body therefore stays outcome-oriented; appendices map each major subsystem to code anchors.

4 Project Management

4.1 Initial V-model phase

The project began with a **V-model style** framing: business requirement gathering with end users in mind, benchmarking comparable products, brainstorming sessions **validated collectively** by the team, and a written requirements package plus an initial Gantt chart. A **technical lead** was appointed to structure the MOE (implementation) track while the project manager covered the MOA (needs, prioritization, specification) track until the specification baseline stabilized.

4.2 Transition to an agile-inspired workflow (Jira, Scrum practices)

After an intensive early phase (approximately **two months** of V-model-oriented execution on a **four-month** calendar), the team concluded that strict phase separation slowed integration of new gameplay and infrastructure ideas. The organization shifted toward an **iterative, Scrum-inspired workflow**: **Jira** for epics, user stories, and time-boxed sprints; backlog ordering by **value vs. risk**; twice-weekly synchronization meetings for decisions, unblockers, and demo-ready increments. The project manager studied agile practice material (including *Scrum Life*) to align rituals with the team's reality rather than copying a textbook process wholesale.

4.3 Planning, coordination, and reporting

Coordination relied on: recurring meetings (~twice per week), shared technical documentation under Docs/, internal validation sessions, and continuous integration feedback via `.gitlab-ci.yml` at the repository root. Work was distributed dynamically across frontend, backend, and infrastructure tasks because multiplayer features require tight coupling between UI state, socket events, and persistence.

4.4 Product management mechanics

Backlog prioritization balanced user-visible value (new game modes, social features) against enabling work (Redis adapter, rate limits, recovery jobs, observability). Cross-cutting concerns (security, idempotency, anti-cheat) were escalated early to avoid late integration risk.

5 Development Tools and Technologies

5.1 Frontend and backend

Client: React 19, TypeScript, Vite 7, Tailwind CSS, Radix UI primitives (`client/src/components/ui/`), react-router-dom v7, Redux Toolkit with RTK Query (`client/src/services/api.ts`), socket.io-client (`client/src/services/socket.ts`), i18next (`client/src/i18n/config.ts`).

Server: Node.js (ESM), Express 4, Socket.IO 4, Prisma 6 with PostgreSQL (`server/prisma/schema.prisma`), Redis clients for rate limiting, action logging, and the Socket.IO Redis adapter (`server/src/index.ts`), Zod validation, JWT auth, bcrypt password hashing, sanitize-html where applicable.

5.2 Database and infrastructure

Local PostgreSQL via `database/docker-compose.yml`. Redis for multi-instance coordination and abuse controls. Optional OpenTelemetry and Prometheus-oriented metrics endpoints (`server/src/observability/`).

5.3 Version control and collaboration

Git repository with GitLab CI configuration at `.gitlab-ci.yml`. Documentation and logic notes live under `Docs/` (including `docs/ARCHITECTURE.md` and `Docs/detail_logique/`).

5.4 Testing and deployment

Client: Vitest, Playwright (`client/e2e/`). Server: Jest and Supertest (`server/src/__tests__/_/`). Desktop/mobile packaging: Electron and Capacitor scripts in `client/package.json`.

6 Use of Artificial Intelligence

6.1 In-product AI: poker bots

The shipped product includes **heuristic poker bots** with configurable difficulty. Core decision logic lives in `server/src/logic/botAI.ts`, orchestrated HTTP services in `server/src/services/botAi.service.ts`, and practice-mode integration (`server/src/poker/services/practiceBotTurns.service.ts`). Difficulty is part of the validated API contract (`server/src/routes/bot.routes.ts`).

6.2 AI-assisted engineering (tooling)

Team members used modern **coding assistants** (e.g., IDE-integrated LLMs) for drafting, refactors, and documentation. Generated output was treated as **untrusted**: reviewed in merge requests, validated by automated tests, and rejected when it contradicted server authority or security invariants.

6.3 Limits and verification

Bots are **not** opaque learned models in production: they are inspectable code paths. Assistant-generated snippets were never merged without human review and test signal.

7 Platform Functionalities

Table 1 lists major domains. Details appear only once, in the referenced appendix, to prevent duplication. **Appendices A–M** also record implemented formulas: closed-form **slot RTP** and unified **roulette RTP**, Monte Carlo equity with an error bound for the **Quantum HUD**, **Quantum Combi** exact probability plus its place in the hand-ranking cascade, and the source files where each is coded.

Table 1: High-level functional inventory (representative surfaces).

Domain	User-visible capability	Primary UI entry	Deep dive
Poker cash	Multiplayer tables, pot logic, chat, actions	<code>/game,</code> <code>/waiting-room</code>	Appendix B
Poker bots	Practice tables, four difficulties	<code>/bot-configuration</code>	Appendix C
HUD	Win chance and hand category estimates	In-game HUD	Appendix D
Hidden bets	Side markets, pricing, settlement	In-game panel, <code>/results</code>	Appendix E
Blackjack	Solo + multi lobby/table	<code>/blackjack,</code> <code>/blackjack/</code> lobby, <code>/blackjack/</code> <code>table/:gameId</code>	Appendix G
Casino	Slot and roulette	<code>/minigames</code>	Appendix H
Tournaments	Waiting, play, results, winner side bets	<code>/tournaments/..</code> .	Appendix F
Social	Friends, invites, loans	<code>/friends,</code> lobby flows	Appendix I
Progression	XP, badges, daily challenges, login streak, leaderboard	<code>/profile,</code> <code>/leaderboard</code>	Appendix J
Account	Auth, 2FA, avatars, profile edit	<code>/auth,</code> <code>/edit-profile</code>	Appendix K
Admin	Operator web console (JWT admin role)	<code>/admin/console</code>	Appendix L
Ops	Metrics, health, CI, packaging	N/A (infra)	Appendix L
Other UX	Gift codes, free recharge, post-game rating modal	<code>client/src/</code> <code>components/</code> <code>Layout.tsx,</code> lobby	Appendix L

Domain	User-visible capability	Primary UI entry	Deep dive
Feedback	User feedback and player reports	Lobby/settings flows	Appendix L
Tournament UX	ZipRush mini-game while waiting	<code>client/src/features/tournament/components/ZipRushMiniGame.tsx</code>	Appendix F
Demo	Example and deal teaching routes	<code>/game-example, /game-deal</code>	Appendix B
A11y	Accessibility, table theming, ambient audio	<code>client/src/contexts/AccessibilityContext.tsx, TableThemeContext.tsx, MusicContext.tsx</code>	Appendix A

8 System Architecture

8.1 Monorepo layout

The repository splits into `client/` (Vite SPA), `server/` (Express + Socket.IO gateway), and `database/` (local Postgres). Client bootstraps providers in `client/src/main.tsx` and routes in `client/src/App.tsx`. Server composes middlewares and REST mounts in `server/src/index.ts`, then attaches `server/src/sockets/game.gateway.ts` to the HTTP server.

8.2 Real-time communication

Real-time communication is one of the central parts of Quantum Bluff. A poker or blackjack game cannot work correctly if players have to refresh the page after every action. Bets, folds, turns, invitations, waiting-room changes and tournament assignments must appear immediately for all concerned users. For this reason, the project uses Socket.IO on the `/socket.io` path, while still keeping REST endpoints for durable actions such as authentication, profile updates, friendship operations or tournament registration.

The most important design choice is the **authoritative server** model. In simple terms, the browser is allowed to say “I want to do this action”, but it is never allowed to decide that the action is valid. For example, the client can request a call, a fold, a table join, or a blackjack move, but the server verifies the user identity, the current game state, the turn order, the chip balance, the table capacity and the game rules before accepting it. This is essential in a card platform because private cards, pot distribution, hidden side bets and player balances must not depend on code running inside the user’s browser.

The socket authentication flow follows the same identity logic as the REST API. A player sends a JWT either in `handshake.auth.token` or through an `Authorization: Bearer ...` header. This token is validated by `server/src/middleware/socketAuth.middleware.ts` and then rechecked in important gateway paths inside `server/src/sockets/game.gateway.ts`. The server rejects malformed tokens, revoked tokens, invalid issuer/audience/type values, admin tokens used on player sockets, missing database users and suspended accounts. Only after these checks does the socket receive a trusted `socket.userId`. This matters because later real-time events do not blindly trust a `playerId` sent by the client; they compare it with the authenticated socket identity.

Socket.IO rooms are used to deliver events only to the users who need them. Each connected user joins a private room named `user:{userId}`, which is used for personal notifications such as friend requests, invitations, warnings, tournament assignments and moderation-related messages. Game tables also have their own rooms, so a poker table update is broadcast to the table participants rather than to every connected browser. Tournaments use rooms such as `tournament:{id}` for roster updates, starts, next rounds, cancellations and final results. This keeps communication organized and avoids mixing unrelated users’ events.

Game synchronization is based on two complementary mechanisms: **events** and **snapshots**. Events are useful for immediate changes, for example “a player joined”, “a player acted”, “the round advanced”, or “the roster changed”. However, relying only on small

incremental events is fragile: if a user refreshes the page, temporarily loses connection, or misses one event, the local UI can become inconsistent. To avoid this, the gateway sends a complete sanitized snapshot when a user joins or reconnects. For poker, live runtime controllers are tracked through `server/src/shared/activeGames.ts`, and serializable snapshots are stored through `server/src/shared/pokerStateStore.ts` and `server/src/poker/services/pokerStateSync.service.ts`. For blackjack, equivalent recovery and snapshot logic exists under `server/src/blackjack/recovery/` and `server/src/blackjack/services/`.

The word *sanitized* is important. A snapshot is not just a copy of the server memory. It is a version of the state prepared for a specific recipient. A seated poker player can receive their own private cards; other players and spectators cannot. A user can see public table state, pot information and visible cards, but hidden information remains hidden. This gives the user interface enough data to render the game while preserving confidentiality and fairness.

Real-time communication is also used outside the game table. The lobby receives live information about rooms, friends, invitations and daily activity. The friends system can notify a user immediately when a friend request is received or accepted. The blocked-user warning flow also uses real-time communication: if a user tries to join a room containing a blocked member, or if a blocked member joins after them, the client receives a warning and can choose whether to continue. This shows that Socket.IO is not only used for cards; it supports the whole multiplayer experience around the game.

Redis support allows the Socket.IO layer to work beyond a single Node.js process. The adapter configured in `server/src/config/socketIoRedis.ts` lets events be propagated across instances, which is important for deployment scenarios where multiple server processes may handle different users. For local development or degraded conditions, the system can still run with a local fallback, but the production-oriented design is ready for distributed event delivery.

Overall, the real-time solution is designed around three principles: the server remains the source of truth, each user receives only the state they are allowed to see, and reconnection is treated as a normal situation rather than as an exceptional failure. This makes the application understandable for users, safer against modified clients, and more stable during real multiplayer sessions.

8.3 Persistence and integrity

Prisma models capture users, games, tournaments, loans, hidden bet tickets, wallet ledger rows, and more (see `server/prisma/schema.prisma`). Critical monetary flows favor server-side orchestration and short transactions.

8.4 Data model overview (Prisma)

The persistence layer is relational PostgreSQL accessed through Prisma. Representative entities include: **User** (credentials, profile, chips, XP fields, avatar storage keys), **Game/seat** models for poker tables, **BlackjackRoom** and related seat/invitation models for multiplayer

blackjack, tournament tables for bracket state and winner-bet tickets, `Friendship` and messaging hooks, loan and ledger tables for social/casino flows, and audit-friendly structures for wallet history. Full cardinality and migration history are only authoritative in `server/prisma/schema.prisma` and the migration folder.

8.5 Diagram

Figure 1 summarizes the request path. For a publication-quality diagram, export a Mermaid figure from `docs/ARCHITECTURE.md` to PNG and include it with `\includegraphics`.

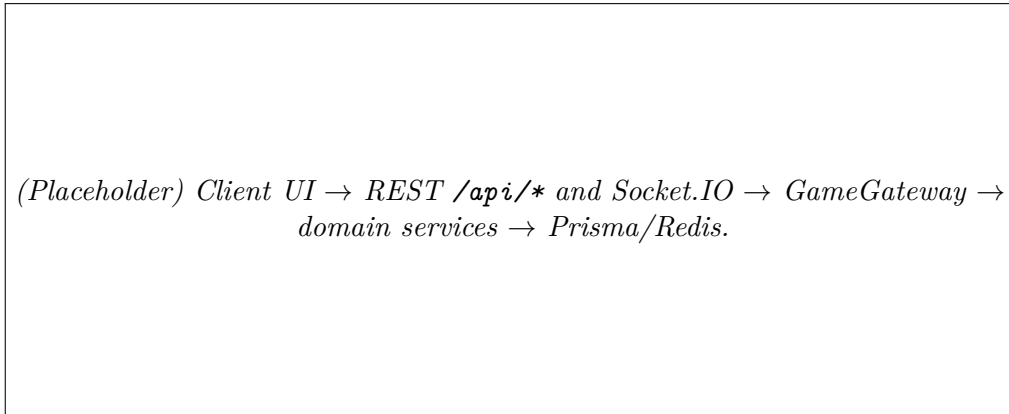


Figure 1: High-level runtime flow (placeholder box; replace with exported diagram if desired).

9 Deployment and operational runbook

9.1 Local development stack

The root script `npm run dev` (see `package.json`) launches PostgreSQL through `database/docker-compose.yml`, a lightweight helper script (`scripts/dev-ai.js`), the Node/Express server (`server/`), and the Vite client (`client/`) under concurrently. Redis should be available for rate limiting, action logging, and the Socket.IO Redis adapter when exercising multi-instance paths (`server/src/config/socketIoRedis.ts`). First-time setup, tester onboarding, and troubleshooting are documented in `README.md`, `Docs/SETUP_TESTEUR.md`, and `Docs/QUICK_TEST_GUIDE.md`.

9.2 Production and desktop delivery

VM-oriented deployment (Node, PostgreSQL, Redis, PM2), environment files, and Electron auto-update mechanics are described in `Docs/DEPLOY.md`. Mobile/desktop packaging scripts are exposed from `client/package.json` (Capacitor and Electron targets). Reverse-proxy samples appear under `nginx/` when applicable.

10 Security, Testing and Stability

10.1 Authentication and anti-cheat

Authentication is the part of the system that answers a simple question: “who is this user, and are they allowed to do this?” In Quantum Bluff, this question is asked not only on normal HTTP routes, but also on Socket.IO connections and game actions. This is important because a multiplayer game exposes many sensitive operations: joining rooms, betting chips, sending invitations, changing profile data, reporting users and accessing admin information.

The main authentication mechanism uses short-lived JWT access tokens generated by `server/src/auth/jwt.service.ts`. A JWT is a signed token that proves the server has authenticated the user. In this project, the token must have the expected issuer, audience, signature algorithm and `type: access`. Its subject is aligned with the user id. HTTP routes only accept the standard `Authorization: Bearer ...` format; raw tokens or malformed headers are rejected early by the bearer-token extraction logic. This avoids ambiguous token parsing and makes the API behavior predictable.

Protected REST routes pass through `server/src/middleware/auth.middleware.ts`. This middleware does several checks instead of only decoding the token. It verifies that the token has not been revoked, validates the JWT signature and metadata, rejects admin-role tokens on player routes, checks that the user still exists in PostgreSQL, and blocks suspended accounts with an explicit `ACCOUNT_SUSPENDED` response. This means that deleting or suspending an account is effective even if an old token still exists on the client side.

Passwords are never stored in clear text. Registration and password changes are handled in `server/src/routes/auth.routes.ts`, where passwords are hashed with bcrypt before persistence. The accepted password format is validated with Zod schemas from `server/src/validation/auth.validation.ts`. The policy requires enough length and complexity, including uppercase, numeric and special-character requirements. The goal is not only to reject weak passwords, but also to centralize the rule so registration, reset and profile password changes stay consistent.

The account layer also includes optional TOTP two-factor authentication in `server/src/routes/twofa.routes.ts`. TOTP is the mechanism used by applications such as Google Authenticator or Authy: after entering the password, the user must also provide a short code generated on a trusted device. Pending setup secrets are stored temporarily in Redis, and disabling 2FA requires both the current password and a valid one-time code. This prevents a stolen session alone from silently disabling the second factor.

Logout and token revocation are handled by `server/src/auth/tokenBlacklist.ts`. Instead of storing the raw token in Redis, the implementation stores a hash of the token with an expiration time. This allows the server to reject a token after logout while avoiding unnecessary storage of sensitive token material. An in-memory fallback exists for degraded development conditions, but Redis is the intended shared store when several processes are running.

The same identity model is enforced on WebSocket traffic. `server/src/middleware/socketAuth.middleware.ts` rejects revoked tokens, invalid JWTs, admin tokens, missing

users and suspended accounts before a socket can interact with game rooms. Then, in `server/src/sockets/game.gateway.ts`, sensitive events compare the trusted `socket.userId` with any player id included in the event payload. If a modified client tries to act for another player, the mismatch is refused and logged. This protects actions such as joining a game, reconnecting, accepting invitations or sending poker actions.

Anti-cheat is implemented as a set of layered controls rather than a single magic function. At the HTTP level, `server/src/middleware/antiCheat.middleware.ts` checks whether the authenticated user is currently banned, records the last observed IP address, and delegates suspicious-pattern detection to `server/src/services/antiCheat.service.ts`. This service can raise alerts when several distinct accounts appear from the same non-local IP address or when reaction times are unrealistically fast. Repeated alerts can lead to a temporary suspension through the `bannedUntil` field.

At the real-time game level, `server/src/utils/antiCheat.ts` tracks action bursts. For example, more than eight actions in a three-second window is considered suspicious by default. The poker action handler can refuse excessive actions and record the incident. This is useful because some abuses do not appear during login; they appear during gameplay, when a script or modified client sends actions faster than a normal player could.

The project also protects against social and room-abuse scenarios. Blocked users are filtered from friend search, blocked relationships can trigger warnings before joining rooms, and player reports are persisted so administrators can review them. This connects security with moderation: the system is not only trying to stop technical attacks, but also to give users tools against unwanted interactions.

Additional hardening is applied at the application boundary. `server/src/index.ts` configures Helmet headers and CORS allowlists. Segmented rate limits protect authentication, bot routes, slot, roulette, blackjack, hidden bets and multiplayer blackjack endpoints. Sensitive mutations can use idempotency middleware from `server/src/middleware/idempotency.middleware.ts`, which helps prevent duplicate chip-changing requests when a user retries or refreshes at the wrong moment. Admin routes are separated from player routes and require the appropriate role.

In summary, the security strategy follows a defence-in-depth approach. Each layer checks what it can know best: validation schemas check input shape, authentication middleware checks identity, socket middleware checks connection trust, game services check rule legality, anti-cheat services check suspicious behavior, and admin tools expose reports and alerts for review. For an academic prototype, this gives a strong and coherent security baseline. A public production launch would still require a professional penetration test, secret rotation procedures and long-term monitoring, but the implemented project already covers the main authentication and anti-cheat risks expected in this scope.

10.2 Testing and debugging

Broad automated coverage on server and client; Playwright smoke tests. Additional manual scenarios are documented under `Docs/POKER_SCENARIOS.md` and related files.

10.3 Automated test inventory and CI (reference snapshot)

Table 2 aggregates counts recorded in `Docs/RAPPORT_TESTS.md` (March 2026 snapshot). Teams should re-run Jest and Vitest after major refactors and update the markdown source of truth.

Layer	Runner	Suite files	Tests
Backend (<code>server/</code>)	Jest	12	257
Frontend (<code>client/</code>)	Vitest	6	20
Total automated	—	18	277

Table 2: Automated test inventory (representative snapshot; see `Docs/RAPPORT_TESTS.md`).

GitLab CI configuration at `.gitlab-ci.yml` encodes build and test gates for merge requests. A consolidated validation narrative for bot flows and UI smoke checks appears in `Docs/rapport-validation.md`.

10.4 Performance and stability

Redis-backed Socket.IO adapter, structured logging, Prometheus metrics endpoint, readiness endpoints, graceful shutdown hooks (`server/src/index.ts`).

11 Technical Challenges

Concurrency around shared tables: poker and blackjack runtimes use locking strategies (see `server/src/poker/services/pokerTableLock.service.ts` and blackjack lock services) to prevent split-brain chip mutations.

Recovery after process restarts: blackjack and tournament subsystems implement recovery services invoked at boot (`server/src/index.ts`).

Hidden bet settlement coupling: market resolution must align with poker street transitions; implemented in hidden-bet resolver modules (Appendix E).

Internationalization without clobbering user preference: guest screens (StartScreen, Auth, global loader) force English using `react-i18next` with a fixed `lng: 'en'` override so the global language stored for authenticated use is unchanged (`client/src/pages/Auth.tsx`, `client/src/contexts/LoaderContext.tsx`).

11.1 Implemented solutions

The implemented solution is a complete web platform rather than a single isolated poker prototype. The project was organized into domain modules that share the same runtime architecture: a React and TypeScript client for the interface, an Express API for REST operations, Socket.IO for real-time communication, Prisma with PostgreSQL for durable storage, and Redis for coordination features such as token revocation, rate limiting and distributed socket events. This structure makes the project easier to understand: the client displays and sends user intentions, while the server validates, computes, persists and broadcasts the result.

For the poker core, the solution is server authority. The server owns the real table state, including seats, blinds, current street, legal actions, pots, side pots, showdowns and chip movements. The client does not decide who wins, how much a player receives, whether an action is legal, or whether a hidden side bet is valid. Poker actions pass through the gateway and `server/src/poker/services/pokerActionOrchestrator.service.ts`; the table logic itself is implemented under `server/src/logic/`. This gives the game a clear trust model: the browser is an interface, not the referee.

Hidden bets were implemented as a separate server-side subsystem under `server/src/poker/hiddenBets/`. This avoids mixing side-market logic directly into the main poker table. Pricing, placement, settlement and ledger effects are handled by dedicated services, and the resolver follows poker street transitions. This design keeps hidden bets attached to the real poker result while reducing the risk of leaking private information to the client.

For real-time synchronization, the project combines Socket.IO events, rooms, snapshots and recovery stores. Important changes are emitted to the relevant room, but reconnecting clients also receive a full sanitized snapshot. Poker snapshots are serialized through `server/src/poker/services/pokerStateSync.service.ts` and stored through `server/src/shared/pokerStateStore.ts`; active games are tracked in `server/src/shared/activeGames.ts`. Blackjack has its own snapshot and recovery services under `server/src/blackjack/`. This solution addresses a common multiplayer problem: a user who refreshes or reconnects should

not be stuck with an old local state.

For blackjack, the project supports both solo and multiplayer experiences. Solo blackjack is a server-controlled loop where the user plays against the dealer. Multiplayer blackjack uses rooms, seats, invitations, snapshots, recovery and table-specific events. The logic remains separate from poker because blackjack has different rules, different round flow and different UI needs. At the same time, it still follows the same project principles: server-side validation, controlled chip updates and real-time user feedback.

For roulette and slot machine features, the solution was to keep the casino mini-games isolated behind their own logic modules and API routes. The server calculates outcomes, validates stakes, applies caps and updates wallet state. The frontend renders the experience and displays history or results, but it does not determine payouts. This keeps the virtual economy coherent across poker, blackjack, roulette and slots.

For tournaments, the implementation separates the lifecycle into clear phases: creation, registration, waiting room, table assignment, round progression, elimination, completion and reward distribution. The frontend feature package lives under `client/src/features/tournament/`, while server-side orchestration lives under `server/src/tournament/`. Socket.IO rooms such as `tournament:{id}` and `user:{userId}` notify players about roster changes, starts, next rounds, kicks, cancellations and results. Tournament winner side bets are handled by their own services so settlement remains tied to the real tournament outcome.

For the lobby, the implemented solution was to make it the main operational screen of the application. After login, the user can see and join poker rooms, blackjack rooms, tournaments, friends, invitations and daily activity without navigating through a marketing-style landing page. The lobby also integrates social actions such as friend status, direct invitations, blocked-user warnings and live room updates. This makes the application feel like a multiplayer platform rather than a collection of disconnected pages.

For the friends and moderation system, the solution combines persistent REST operations with live notifications. Users can send and receive friend requests, remove friends, block users, unblock them from a blocked list, report players with a reason, open chat interactions and receive warnings when joining rooms containing blocked users. Reports and moderation data are stored server-side so they can be reviewed by administrators. This feature is important because a multiplayer application needs user-safety tools, not only game logic.

For progression and retention, XP, badges, daily challenges, daily login streaks, leaderboards, gift codes and recharge flows were implemented as separate modules. The user can see progress and rewards in the interface, but server-side services remain responsible for eligibility, caps and balance-changing effects. This prevents the progression system from becoming only a visual layer and keeps it connected to the persistent user profile.

For authentication, security and anti-cheat, the solution is intentionally layered. HTTP middleware, socket middleware, gateway checks, validation schemas and domain services all verify different parts of the same trust chain. A forged REST body cannot replace the authenticated JWT identity. A forged socket payload cannot act as another player. A suspended user is blocked during authentication, REST access and socket connection. Suspicious behavior is logged and can lead to anti-cheat alerts or temporary suspension. This

redundancy is useful because security mistakes often happen when only one layer is trusted too much.

For monetary integrity, every chip-changing operation is treated as sensitive. The project uses server-side settlement, Prisma persistence, transaction-oriented patterns, wallet ledger rows, idempotency keys where needed, and domain-specific services for rewards or payouts. This is visible in casino games, hidden bets, tournament rewards, social loans and wallet updates. The shared rule is simple: the balance shown in the browser is only a display; the database and server services are the real source of truth.

For the interface, the solution was to build functional screens around the user's actual workflows: lobby, poker table, blackjack, roulette, mini-games, friends, profile, leaderboard, tournaments and admin console. Reusable components and shared services reduce duplication. Internationalization uses `i18next` resources, and the application includes multiple locales. Accessibility, theming, music/effects settings and responsive layout work were added so the interface is not only technically functional, but also usable in realistic conditions.

For deployment and operations, the project includes Docker-oriented local setup, PostgreSQL, Redis, Nginx-related configuration, GitLab CI and production documentation. Health checks, metrics-oriented code and graceful shutdown behavior are part of the server infrastructure. These elements help move the project from a local student prototype toward something that can be deployed, tested and maintained.

For validation, automated tests cover important pure logic, validation schemas, tournament helpers, gateway behavior, waiting-room identity checks, recovery logic and several API flows. The CI pipeline acts as a regression gate for merge requests. Manual validation documents and scenario files complement automated tests for flows that are difficult to cover completely, such as full multiplayer sessions. The result is not only a feature-complete implementation, but a project with traceability between the implemented modules, the reported architecture and the expected user behavior.

The remaining work before a real public release would be external security auditing, longer load testing, a formal data-protection review, more end-to-end coverage for complete user journeys, and a polished production observability dashboard. For the academic scope of the project, however, the implemented solution covers the main functional, real-time, security, social, economy, gameplay and deployment needs of the platform.

12 Ethics, privacy, and academic integrity

12.1 Virtual economy and regulatory framing

Quantum Bluff uses **in-game virtual chips** for learning and demonstration. It is not a licensed real-money gambling operator; features that resemble casino products are implemented in an **academic engineering** context and should be evaluated as software artefacts, not as a commercial gaming offer.

12.2 Personal data

Accounts store identifiers needed for authentication and gameplay (email, username, hashed password, optional TOTP secrets, profile metadata). Operators deploying a public instance must align hosting, retention, and breach notification with applicable law (including GDPR for EU users). This report does not replace a data protection impact assessment.

12.3 Integrity of authorship and AI assistance

Section 6 states how large language models were used as drafting assistants. All merged code remained subject to human review, tests, and repository governance. Each author is expected to sign off on their contribution statement in Section 13.

13 Individual Contributions

13.1 Omar – project management and architecture alignment

Organizational narrative: Section 4. Beyond ceremonies and backlog arbitration, Omar coordinated **cross-cutting technical commitments**: early adoption of Redis for Socket.IO horizontal scale-out and rate limiting, alignment between Prisma schema changes and gameplay features, and risk reviews before widening scope (tournaments, casino mini-games, loans). Interface with the appointed technical lead focused on sequencing risky merges (socket gateway, recovery jobs) away from demo deadlines.

13.2 Linda – security, real-time features and UI/UX

I worked on backend integration, real-time features, security and UI/UX. My main technical contributions were game routes, blinds configuration, waiting-room fixes, player-state debugging, socket authentication, JWT hardening, stronger password rules, rate limits and anti-cheat checks.

I also worked on the friends and lobby experience: friend search, add-friend modal, friend cards, chat access, remove friend, block/unblock, user reports, invitations, live notifications, activity status and blocked-user warnings. On the UI side, I improved the lobby layout, reduced empty space and scrollbars, and contributed to profile/avatar, poker table, blackjack, roulette, mini-games, settings and responsive layout improvements.

13.3 Other contributors

The repository does not contain signed-off, per-person effort logs, story point assignments, or validated textual attributions for all remaining members. To satisfy examination requirements, the table below must be **completed and countersigned by the team** (replace placeholder cells).

Member	Primary areas (self-declared)	Verifiable artefacts (commits, MRs, modules)
Massi	<i>[to complete]</i>	<i>[to complete]</i>
Abdoulaye	<i>[to complete]</i>	<i>[to complete]</i>
Soheil	<i>[to complete]</i>	<i>[to complete]</i>
Mohamed	<i>[to complete]</i>	<i>[to complete]</i>
Azra	<i>[to complete]</i>	<i>[to complete]</i>
Yigit	<i>[to complete]</i>	<i>[to complete]</i>

Table 3: Individual contributions (template — team must fill before submission).

14 Limitations and Future Work

14.1 Current limitations

- **Documentation volume vs. examiners' time:** internal generated reports (e.g., `Docs/Rapport_final/RAPPORT_TECHNIQUE_COMPLET.md`) are useful as indexes but are not all human-curated line-by-line.
- **Dual loader providers:** `client/src/main.tsx` and `client/src/App.tsx` both wrap `LoaderProvider`; harmless but confusing—should be deduplicated.
- **Packaging surface area:** Electron/Capacitor increase release engineering work (signing, store policies) beyond the core web path.

14.2 Future improvements

Hardening continuous deployment secrets, expanding observability dashboards, deeper bot modeling (still code-first), and richer tournament analytics.

15 Conclusion

Quantum Bluff exceeded its initial poker-only charter by integrating multiple casino modes, tournaments, social lending, progression systems, and operational hardening while keeping a monorepo with clear separation between UI, API, and authoritative game logic. The project demonstrates that a student team can approach industry-like constraints—real-time state, monetary integrity, security, and observability—when management explicitly trades early waterfall rigidity for controlled agile iteration.

Lessons retained for similar efforts: (1) anchor scope changes to **recoverable increments** (feature flags, migrations, recovery jobs); (2) treat generated documentation as **hypotheses** until cross-read against code; (3) keep **security and idempotency** on the critical path rather than as end-of-project polish.

Acknowledgements

The team thanks the University of Strasbourg Computer Science faculty for supervising the L3 capstone framework, the open-source maintainers of the libraries listed in Section 5, and peers who provided feedback during playtests. Replace or extend this paragraph with institution-specific acknowledgements as required by the course booklet.

A Repository map and routing

A.1 Programmed routing and shell

Client routes are declared in `client/src/App.tsx`: public splash and auth (`/`, `/auth`, `/auth/admin`); authenticated gameplay and hub routes under `ProtectedRoute` with `Layout` (lobby, poker, blackjack, tournaments, mini-games, profile, friends, admin console). Admin routes use `AdminProtectedRoute`. Global providers stack in `client/src/main.tsx` (error boundary, loader, Redux, toasts, user, socket, Quantum HUD, hidden bets, audio) with an additional nested loader/invitation shell in `App.tsx`. Server REST mounts, rate-limit tiers, and health/metrics endpoints are wired in `server/src/index.ts`. A narrative overview also exists in `docs/ARCHITECTURE.md`.

B Poker engine and cash game

B.1 What was programmed

Authoritative table logic lives in `server/src/logic/GameTable.ts` (seats, streets, pot and side pots, action legality, showdown ordering) and `server/src/logic/CashGameController.ts` (chip debits/credits, transitions between phases). Hand evaluation and category ordering use `server/src/logic/Evaluator.ts`. Real-time delivery and timers (e.g. `TURN_TIMER` with 20s auto-fold/check) are implemented in `server/src/sockets/game.gateway.ts` together with anti-cheat hooks. The client implements presentation and input in `client/src/pages/Game.tsx` and reusable components under `client/src/components/` (`PokerTable`, `ActionButtons`, `CommunityCards`, `chat`, `HUD` hooks). **All wagering state changes are validated and applied on the server**; the client never “decides” the winner.

B.2 Rules enforced

Texas Hold'em flow from blinds through preflop, flop, turn, river, and showdown; fold, check, call, raise semantics; minimum raise increments; side pots when players are all-in for different amounts. Waiting-room creation and join flows are handled by dedicated routes (`server/src/routes/waitingRoom.routes.ts`, `server/src/routes/game.api.routes.ts`) coordinated with the gateway.

B.3 Further reading

Structured logic notes: `Docs/detail_logique/backend/01_poker_moteur.md`, `Docs/detail_logique/backend/02_cashgame_controller.md`.

C Bot AI and practice integration

C.1 Programmed pipeline

POST `/api/bot/action` (`server/src/routes/bot.routes.ts`) validates difficulty and cards, then calls `server/src/logic/botAI.ts`. Practice tables chain bot turns through `server/src/poker/services/practiceBotTurns.service.ts`; HTTP orchestration and optional expert delegation sit in `server/src/services/botAi.service.ts`. Four difficulties are supported: `easy`, `medium`, `hard`, `expert`.

C.2 Strength signal and decisions

Let H be the scalar hand score returned by `getHandValue` for the bot’s visible cards (two to seven cards). The code normalizes

$$\sigma = \min\left(1, \max(0, H/S_{\max})\right), \quad S_{\max} = 9 \cdot 15^5 + 14 \cdot 15^4,$$

matching the encoding used by `server/src/logic/Evaluator.ts`. Each difficulty combines σ with pot odds (`callAmount`, `potSize`), heads-up vs. multiway flags, and pseudo-random draws to return `FOLD`, `CALL`, `CHECK`, or `RAISE` with an optional raise amount (chips rounded via `intChips`). The easy tier, for example, injects naive bluffs on a free street with fixed probability and biases calls when facing small bets with medium strength.

C.3 Further reading

Design notes: `Docs/IA_EXPERT_BOT.md`.

D Probabilities, Quantum HUD, and Monte Carlo helper

D.1 Division of responsibilities

The **Quantum HUD** (`client/src/components/QuantumHUD.tsx`, context `client/src/contexts/QuantumHUDContext.tsx`) displays *estimates* for the authenticated player. These are computed client-side for responsiveness and are **not** used to settle real-money outcomes (the product uses virtual chips). Server-side hidden-bet pricing uses separate tabular services (Appendix E).

D.2 Monte Carlo equity estimator

Implementation file: `client/src/utls/pokerMonteCarloEquity.ts`. Let N be the number of iterations (default $N = 2000$, constant `MONTE_CARLO_DEFAULT_ITERATIONS`). For each iteration the code:

1. Builds the remaining deck excluding known hero and community cards (`buildDeckExcluding`).
2. Draws, without replacement, the missing board cards and $2 \times n_{\text{opp}}$ opponent hole cards using a partial Fisher–Yates swap sampler (`drawWithoutReplacement`).
3. Evaluates the hero and each opponent seven-card hand with `evaluateSevenEval` (same taxonomy as the server evaluator).
4. Splits the simulated pot equally among all players that achieve the best score.

If $S_t \in \{0, 1/k\}$ is the hero’s share on iteration t when k players tie for the best score, the reported equity is

$$\hat{e} = \frac{1}{N} \sum_{t=1}^N S_t.$$

A histogram of category indices (0–9) is accumulated in parallel for UX badges.

D.3 Monte Carlo error budget (HUD analogue to RTP)

The HUD does **not** implement a house-banked game: there is no monetary **RTP** in the casino sense. The quantity $\hat{e}$ is an **equity** estimate (expected share of the pot at showdown under uniform completion of unknown cards). If e denotes the true value of that expectation under the model, then $\mathbb{E}[\hat{e}] = e$ (each draw is independent and fair with respect to the remaining deck). Because $0 \leq S_t \leq 1$,

$$\text{Var}(S_t) \leq \frac{1}{4}, \quad \text{Var}(\hat{e}) = \frac{\text{Var}(S_t)}{N}, \quad \text{SE}(\hat{e}) \leq \frac{1}{2\sqrt{N}}.$$

With the shipped default $N = 2000$, $\text{SE}(\hat{e}) \leq 1/(2\sqrt{2000}) \approx 0.01118$ (about 1.1 equity points at most in the worst-case variance bound).

D.4 Quantum Combi—exact enumeration formula

The promotional “Quantum Combi” category (documented for players in `Docs/final_report/QUANTUM_COMBI.adoc`) requires ranks $\{3, 5, 6, 7, 10\}$ each to appear **at least once** among the seven cards (two private + five community). All $\binom{52}{7}$ seven-card subsets are equally likely:

$$\binom{52}{7} = 133784560.$$

For each rank r in that set, let A_r be the event “no card of rank r among the seven”; then $|A_r| = \binom{48}{7}$. By the principle of inclusion–exclusion over five ranks,

$$\left| \bigcup_r A_r \right| = \sum_{k=1}^5 (-1)^{k+1} \binom{5}{k} \binom{52-4k}{7} = 133002736.$$

Hence the favourable count is $133784560 - 133002736 = 781824$ and

$$P(\text{Quantum Combi}) = \frac{781824}{133784560} \approx 0.00584 \approx 0.584\%.$$

D.5 Quantum Combi in the hand-ranking ladder (server and HUD)

Both `server/src/logic/Evaluator.ts` and `client/src/utils/pokerHandCategory.ts` use the same **best-of-seven** cascade. Categories are integers 0 (high card) through 9 (straight flush); a larger packed score beats a smaller one. After ruling out straight flush (9), four-of-a-kind (8), and full house (7), the code tests **Quantum Combi before flush or straight**. Quantum applies iff every rank in $\{3, 5, 6, 7, 10\}$ appears at least once in the seven rank values (Ace is coded as 14). Let V be the multiset of those seven values; subtract one occurrence of each mandatory rank; sort the remaining values descending and take the top two as kickers (κ_1, κ_2) . The score is

$$\text{packScore}(6, \kappa_1, \kappa_2) = 6 \cdot 15^5 + \kappa_1 \cdot 15^4 + \kappa_2 \cdot 15^3,$$

padding unused kicker slots with zeros in the implementation. Only if Quantum fails does the evaluator continue to flush (5), straight (4), three-of-a-kind, two pair, one pair, high card. Thus Quantum is positioned **strictly between full house and flush**, as a design choice documented in `Docs/final_report/QUANTUM_COMBI.adoc`.

E Hidden betting system

E.1 Programmed architecture

Hidden side markets are implemented under `server/src/poker/hiddenBets/`: typed markets and snapshots (`markets.ts`, `types.ts`), pricing tables (`pricing/pricingTables.ts`, `pricingTables.v1.ts`), live heuristics (`pricing/livePricing.ts`), quote and placement services, ledger context, and street-aware resolvers (`resolver/hiddenBetResolver.ts`). HTTP routes are mounted in `server/src/routes/hiddenBets.routes.ts`.

E.2 Odds and tables (example)

Live markets use tabulated decimal odds keyed by table parameters. For instance, `oddsPlayerWinsCurrentHand` maps the number of active players $n \in [2, 9]$ to a fixed grid (e.g. $n = 2 \rightarrow 1.9$, $n = 5 \rightarrow 3.0$, $n = 9 \rightarrow 4.6$); companion functions cover showdown vs. fold endings. These values are **heuristic** prices distanced from the pre-hand model (see code comments `hidden-bets-live-v1`), not Monte Carlo outputs from the HUD.

E.3 Further reading

`Docs/intern/HIDDEN_BETS_SPEC.md`, `Docs/detail_logique/backend/07_hidden_bets_moteur.md`.

F Tournaments and winner side bets

F.1 Programmed modules

Tournament lifecycle (registration, waiting room, table assignment, blind escalation, elimination, results) is implemented under `server/src/tournament/` with recovery jobs invoked at server boot (`server/src/index.ts`). Winner-side markets reuse wallet patterns in `server/src/tournament/winnerBets/tournamentWinnerBet.service.ts`. The client feature pack lives in `client/src/features/tournament/` (waiting room, in-tournament table, results, winner-bet panel). A lightweight `ZipRushMiniGame` retains players during queue phases.

F.2 Further reading

`Docs/intern/TOURNAMENT_FLOW_REAL_VS_EXPECTED.md`.

G Blackjack solo and multiplayer

G.1 Solo blackjack engine

Authoritative rules are implemented in `server/src/logic/blackjack.ts`. The server uses a six-deck shoe (`DECKS = 6`), shuffled in-place with Fisher–Yates (`shuffleShoe`). Card values follow standard rules: faces and tens count as 10, aces count as 11 when possible without exceeding 21, otherwise as 1. The dealer draws until the hard/soft total is at least 17; because aces are valued optimistically, the dealer **stands on soft 17** (e.g. A+6). Natural blackjack pays a 3:2 bonus on the stake (implemented as integer chip rounding per `intChips`); other wins pay 1:1; pushes return the stake. Double down is permitted only with exactly two player cards and issues one extra card.

Solo routes: `server/src/routes/blackjack.routes.ts`. Multiplayer rooms and seats: `server/src/routes/blackjackMulti.routes.ts`, controllers under `server/src/logic/BlackjackTableC` `ts`, in-memory runtime maps with recovery under `server/src/blackjack/`. UI: `client/src/pages/Blackjack.tsx`, lobby/table flows, components in `client/src/components/blackjack/`.

G.2 Further reading

`Docs/detail_logique/backend/03_blackjack_solo_moteur.md`, `Docs/detail_logique/backend/04_blackjack_multi_moteur.md`.

H Slot machine, roulette, and casino wallet

H.1 Three-reel slot model

The server module `server/src/logic/slotMachine.ts` draws three independent reels. Symbol weights are

$$w_{\text{cherry}} = 32, w_{\text{lemon}} = 24, w_{\text{bell}} = 18, w_{\text{seven}} = 14, w_{\text{diamond}} = 8, \quad W = \sum_i w_i = 96.$$

One reel satisfies $\Pr[R=i] = w_i/W$. Triples satisfy $\Pr(R_1=R_2=R_3=s) = (w_s/W)^3$. Triple s pays total return multiplier $m_s \in \{5, 8, 10, 15, 20\}$ on the stake; an **exact pair** (two equal symbols, third different) pays multiplier 1 (stake returned, zero net profit); otherwise the payout multiplier is 0. RNG is injectable (`RandomIntFn`, default `crypto.randomInt` via `server/src/rng/rng.service.ts`).

H.2 Complete slot RTP (closed form, current constants)

Let $X = \text{winAmount}/\text{bet}$ be the **gross return multiplier** (total coins returned by the machine, including the stake on a push). Write R_1, R_2, R_3 for the three reel symbols.

Triple (three of a kind).

$$\Pr(\text{triple of } s) = \left(\frac{w_s}{W}\right)^3, \quad \Pr(\text{any triple}) = \sum_s \left(\frac{w_s}{W}\right)^3 = \frac{\sum_s w_s^3}{W^3}.$$

Numerically $\sum_s w_s^3 = 32768 + 13824 + 5832 + 2744 + 512 = 55680$ and $W^3 = 96^3 = 884736$, hence

$$\Pr(\text{any triple}) = \frac{55680}{884736} \approx 0.0629 \text{ (6.29\%)}. \quad \text{}$$

Exact pair (not a triple). For a doubled symbol A ,

$$\Pr(\text{pair with } A \text{ doubled}) = \frac{3 w_A^2 (W - w_A)}{W^3}, \quad \Pr(\text{any pair}) = \sum_A \frac{3 w_A^2 (W - w_A)}{W^3}.$$

Numerically $\sum_A 3 w_A^2 (W - w_A) = 461952$, so $\Pr(\text{pair}) = 461952/884736 \approx 0.5221$ (52.21%).

Lose.

$$\Pr(\text{lose}) = 1 - \Pr(\text{triple}) - \Pr(\text{pair}) \approx 0.4149 \text{ (41.49\%)}. \quad \text{}$$

Expected multiplier and RTP. Because outcomes partition $\{\text{lose, pair, triples}\}$,

$$\mathbb{E}[X] = \sum_s \left(\frac{w_s}{W}\right)^3 m_s + \Pr(\text{pair}) \cdot 1 = \frac{\sum_s m_s w_s^3 + 461952}{884736}.$$

Here $\sum_s m_s w_s^3 = 5 \cdot 32768 + 8 \cdot 13824 + 10 \cdot 5832 + 15 \cdot 2744 + 20 \cdot 512 = 384152$, so

$$E[X] = \frac{384152 + 461952}{884736} = \frac{846104}{884736} \approx 0.9563.$$

The **return-to-player (RTP)** expressed as a percentage of the stake is $100 E[X] \approx 95.63\%$. The **house edge** on the stake is $1 - E[X] \approx 4.37\%$. The expected **net** gain per spin is $E[G] = E[\text{winAmount} - \text{bet}] = \text{bet} (E[X] - 1) \approx -0.0437 \times \text{bet}$.

Variance (optional diagnostic). With the same X , the AsciiDoc reference reports $\text{Var}(X) \approx 3.12$ (standard deviation ≈ 1.77); hence $\text{Var}(G) = \text{bet}^2 \text{Var}(X)$ for a flat bet. See `Docs/SLOT_MACHINE.adoc` for the full derivation.

H.3 European roulette model

The server module `server/src/logic/roulette.ts` draws a single winning integer uniformly from $\{0, \dots, 36\}$ (`crypto.randomInt` through the same RNG service). Payouts follow the European **total-return** multipliers on the stake for winning pockets (straight 36, split 18, street 12, corner 9, six-line 6, dozen/column 3, even-money 2). Zero causes outside bets to lose unless covered.

H.4 Complete roulette RTP (all proportional inside bets)

Consider any bet line that covers exactly k distinct winning pockets ($1 \leq k \leq 18$ on the proportional ladder used in code) with European gross multiplier $m = 36/k$ applied to the stake when one of those pockets hits. Because each pocket has probability $1/37$,

$$E\left[\frac{\text{payout}}{\text{bet}}\right] = \frac{k}{37} \cdot m = \frac{k}{37} \cdot \frac{36}{k} = \frac{36}{37} \approx 0.97297.$$

Hence every such line has the same **RTP** $= 36/37 \approx 97.297\%$ of the stake returned as gross payout in expectation, and the same **house edge** $= 1/37 \approx 2.703\%$ on the stake. The expected **net** gain per unit staked on a single line is

$$\frac{k}{37} \cdot m - 1 = -\frac{1}{37}.$$

When several independent bet lines are placed on one spin, expectation adds across lines; the server enforces caps (`ROULETTE_MAX_BET_CAP`, `ROULETTE_MAX_TOTAL_STAKE`, `ROULETTE_MAX_BETS_PER_SPIN`) before settlement. Wheel animation order is `EUROPEAN_WHEEL_ORDER`. Full payout tables: `Docs/final_report/ROULETTE.adoc`.

H.5 Wallet and ledger

Casino flows integrate with wallet/ledger helpers under `server/src/casino/` and `server/src/wallet/`. Additional narrative: `Docs/detail_logique/backend/05_roulette_moteur.md`, `Docs/detail_logique/backend/06_slot_moteur.md`, `Docs/final_report/ROULETTE.adoc`.

I Social graph, invitations, and peer loans

I.1 Programmed surfaces

Friends UI: `client/src/pages/Friends.tsx`. REST: `server/src/routes/friends.routes.ts`; poker invitations `server/src/routes/invitation.routes.ts` (also under `/api/invitations`); blackjack room invitations share patterns with the lobby. Peer loans: `server/src/routes/friendLoan.routes.ts`, business logic in `server/src/services/friendLoan.service.ts`, real-time updates via `server/src/services/friendLoan.emit.ts` and socket listeners on the client.

I.2 Further reading

`Docs/detail_logique/backend/09_friend_loans_moteur.md`.

J Progression, challenges, and leaderboards

J.1 Programmed logic

Shared rules in `server/src/logic/gamification.ts` (XP curves, badge triggers). Daily challenges and login streak are isolated modules (`server/src/dailyChallenges/`, `server/src/dailyLogin/`) with dedicated routers. Public leaderboards: `server/src/routes/leaderboard.routes.ts`; client pages under `/leaderboard` and profile summaries.

J.2 Further reading

`Docs/detail_logique/backend/10_gamification_moteur.md`.

K Authentication, profiles, and i18n

K.1 Programmed security and profiles

JWT issuance and verification on `server/src/routes/auth.routes.ts`; password hashing (bcrypt) and validation schemas (Zod). Optional TOTP 2FA: `server/src/routes/twofa.routes.ts`. Client auth flows: `client/src/pages/Auth.tsx`. Profile and avatar uploads: `client/src/pages/EditProfile.tsx` with binary fields or URLs on the Prisma `User` model. Internationalization bundles live in `client/src/i18n/locales/` (five languages); guest-only screens force English overlays where documented in Section 11.

L Operations, Redis, CI, and packaging

L.1 Programmed operations

Redis clients for Socket.IO horizontal scale-out: `server/src/config/socketIoRedis.ts`. Rate limiting and action logs share Redis primitives configured in `server/src/index.ts`. Observability helpers: `server/src/observability/`. CI pipeline: `.gitlab-ci.yml`. Desktop/mobile packaging scripts in `client/package.json` (Electron, Capacitor). Admin web console: `client/src/pages/AdminConsole.tsx` with `server/src/routes/adminConsole.routes.ts`. Dangerous admin/dev routes are compiled out or gated when `env.isProduction` is true.

M External references and compendium

- Initial formal requirements PDF: Docs/Cahier_de_charge_6A_PI.pdf (not embedded here).
- Auto-generated wide-coverage technical compendium: Docs/Rapport_final/RAPPORT_TECHNIQUE_COMPLET.md (use as index; verify claims against code).
- Security dossiers: Docs/Architecture/SECURITE_A_TO_Z.md, Docs/security/.
- HTTP and socket maps: Docs/detail_logique/network/01_http_api_map.md, Docs/detail_logique/network/02_socket_events_map.md.
- Combination analysis report: Docs/final_report/QUANTUM_COMBI.adoc.
- Slot RTP and outcome algebra (AsciiDoc): Docs/SLOT_MACHINE.adoc.
- Roulette payouts and expectation: Docs/final_report/ROULETTE.adoc.

References

- [1] Meta. *React documentation* (React 19). <https://react.dev/> (accessed May 2026).
- [2] Microsoft. *TypeScript handbook*. <https://www.typescriptlang.org/docs/> (accessed May 2026).
- [3] Vite contributors. *Vite guide*. <https://vite.dev/guide/> (accessed May 2026).
- [4] OpenJS Foundation. *Node.js runtime documentation*. <https://nodejs.org/docs/latest/api/> (accessed May 2026).
- [5] OpenJS Foundation. *Express web framework*. <https://expressjs.com/> (accessed May 2026).
- [6] Socket.IO project. *Socket.IO server and client (v4)*. <https://socket.io/docs/v4/> (accessed May 2026).
- [7] Prisma Data Inc. *Prisma ORM documentation*. <https://www.prisma.io/docs> (accessed May 2026).
- [8] PostgreSQL Global Development Group. *PostgreSQL documentation*. <https://www.postgresql.org/docs/> (accessed May 2026).
- [9] Redis Ltd. *Redis documentation*. <https://redis.io/docs/> (accessed May 2026).
- [10] Redux team. *Redux Toolkit and RTK Query*. <https://redux-toolkit.js.org/> (accessed May 2026).
- [11] i18next authors. *i18next documentation*. <https://www.i18next.com/> (accessed May 2026).
- [12] Zod maintainers. *Zod schema validation*. <https://zod.dev/> (accessed May 2026).
- [13] OpenJS Foundation. *Jest testing framework*. <https://jestjs.io/> (accessed May 2026).
- [14] Vitest contributors. *Vitest*. <https://vitest.dev/> (accessed May 2026).
- [15] Microsoft. *Playwright end-to-end testing*. <https://playwright.dev/> (accessed May 2026).
- [16] Tailwind Labs. *Tailwind CSS documentation*. <https://tailwindcss.com/docs> (accessed May 2026).
- [17] Schwaber, K., Sutherland, J. *The Scrum Guide* (2020). <https://scrumguides.org/scrum-guide.html> (accessed May 2026).
- [18] GitLab Inc. *GitLab CI/CD documentation*. <https://docs.gitlab.com/ee/ci/> (accessed May 2026).
- [19] Université de Strasbourg. *Faculté des mathématiques et informatique*. <https://mathinfo.unistra.fr/> (accessed May 2026).